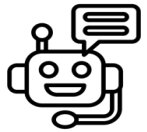
Agentic Personal Assistant & MCP

Class four

Types of AI-Powered Solutions

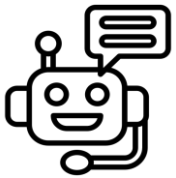
- Workflow 
  - Automated **process pipeline**

- Intelligent Chatbot 
  - The system could perform **iterative queries and analysis** based on the end user queries

- Agentic Assistant 
  - It will understand, plan and execute the tasks **autonomously**

What is AI Agent

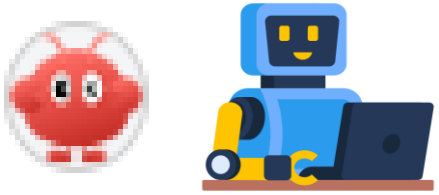
From "answering questions" to "getting things done for you"



Traditional AI Solutions

- Answers your questions
- Waits for your instructions
- One step at a time
- You do most of the work





AI Agent

Brain (LLM)
Memory
Tools
Loop (iteration)

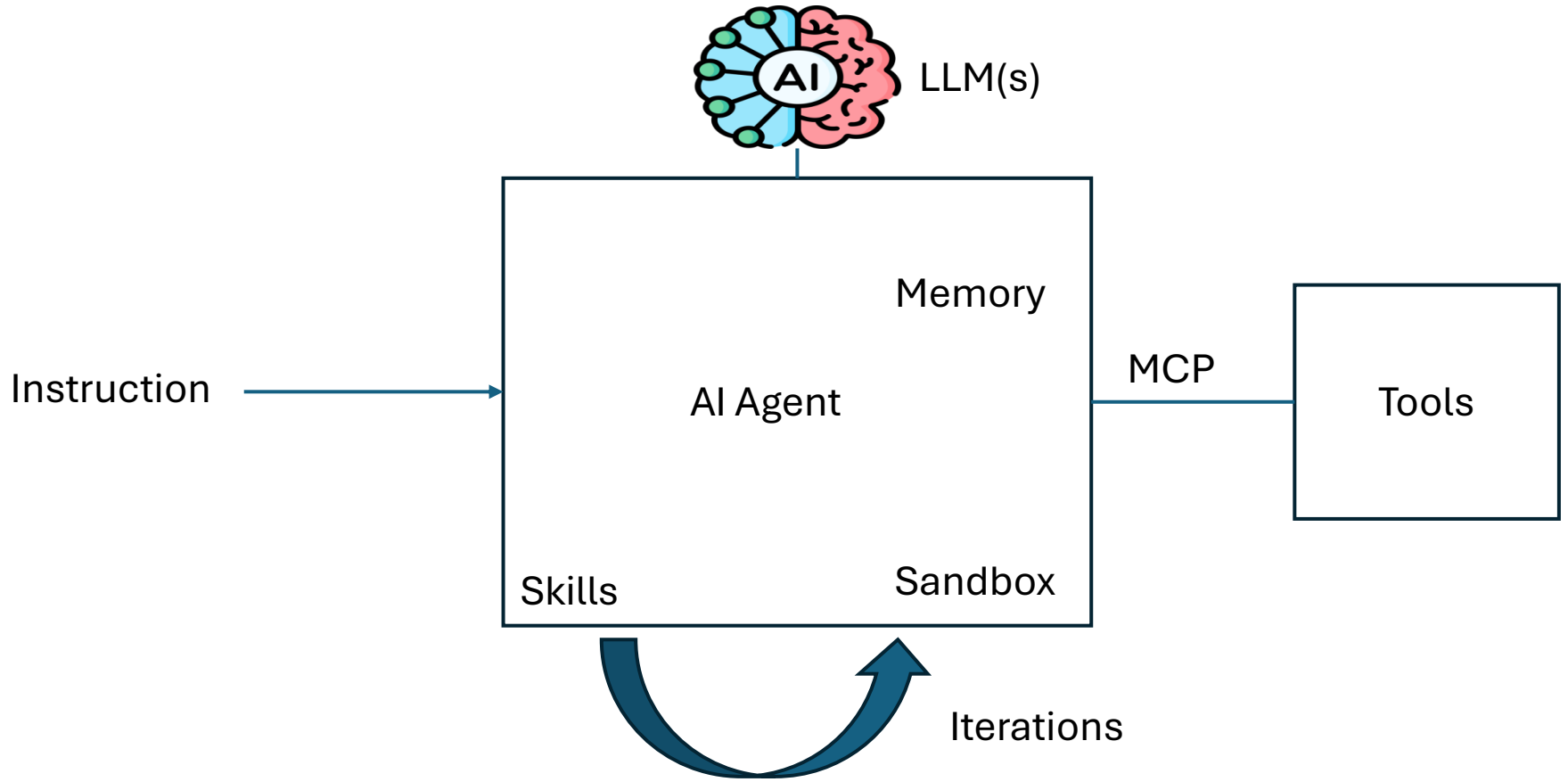
- Understands your goal
- Plans the steps itself
- Uses tools (email, calendar, web, apps)
- Takes action and finishes the task
- Learns and adapts as it goes

Saves you time Reduces your effort Gets better results Frees you up

Important Consideration (Risk)

- AI misunderstand the goal and make mistake
- Protect your privacy-Be careful what data you want to share
- Deploy Guardrail with 7x24 monitoring

Agentic Development Framework



AI Agent=LLM Model + Harness

How Agents Work — Two Core Patterns (LLM Reasoning Patterns)

ReAct

Reasoning + Acting

Think → Act → Observe → Repeat

- Interleaves thinking and action
- Highly adaptive to new results
- Best for exploratory / dynamic tasks
- Can get expensive on long chains

Plan-and-Execute

Plan first, then execute

Create full plan → Run steps → Replan if needed

- Planner creates a complete roadmap
- Executor carries out each step
- Better for long, structured tasks
- More predictable cost & structure

What is a Harness?

Harness = the engineering layer that makes the Agent reliable

Analogy

LLM = the horse (raw power & intelligence)
Harness = the bridle & gear (control, direction, safety)

Format Cleaning

Force clean structured output (e.g. valid JSON)

Parameter Validation

Check tool arguments before execution

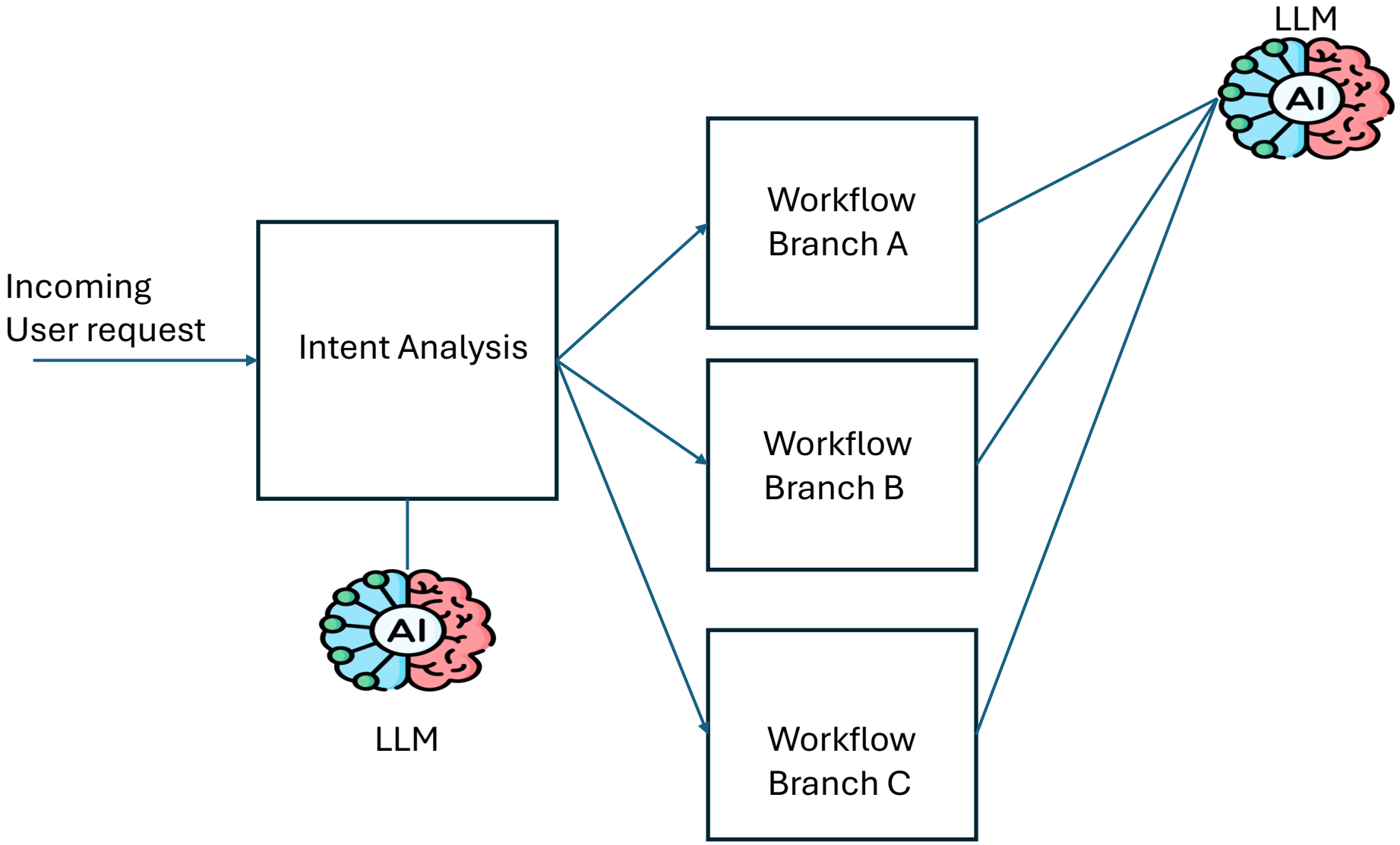
Input / Output Filtering

Guard against injection & bad content

Code-level Corrections

Hard rules that fix recurring mistakes

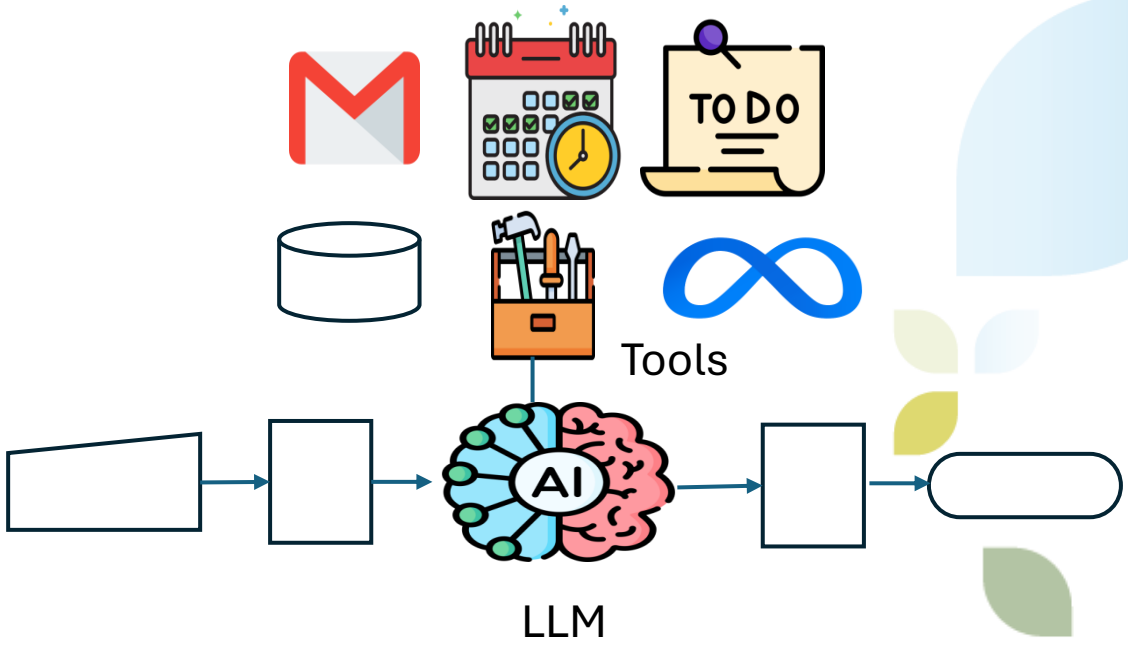
Recap: Workflow with Routing



Agentic Assistant

An AI System which could Plan and Act **Autonomously**

[User Goals]
↓
[Planner)] → Break down the task steps
↓
[Reasoner)] → Choose the appropriate action
↓
[Executor)] → Run the command or API
↓
[Memory] → [Knowledge Base)]
↓
[Feedback Loop] → Evaluate the results and improve



Example of Agentic Assistant



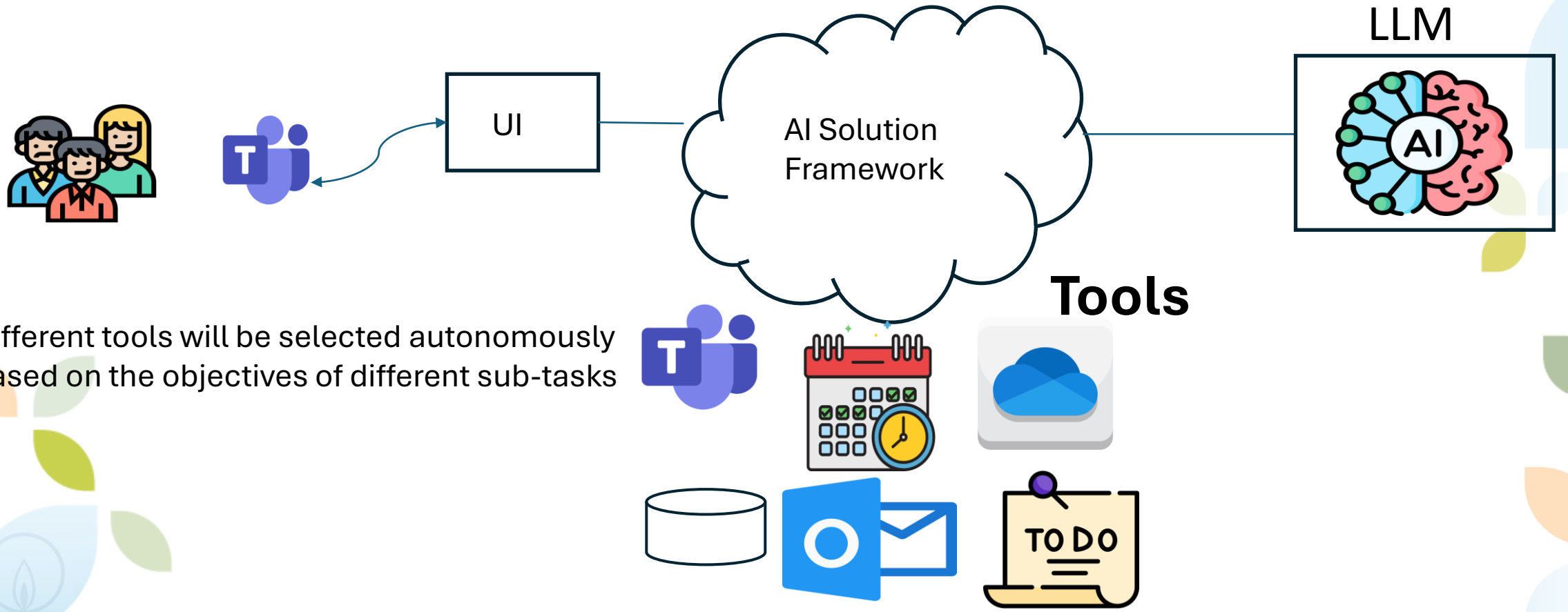
Send email to David and ask for the project update.
Also send the meeting invite to him on next Friday 15:00

→

Task 1: Send an email to David requesting a project update.
Task 2: Send a meeting invite to David for next Friday at 15:00.

→

Select the required tools and plan/execute the tasks accordingly



Different tools will be selected autonomously based on the objectives of different sub-tasks

Algo Trading System with AI Agent

2. Propose the corresponding trading strategy

3. Approve the execution of the trading strategy ?

4. Execute the order

Construct
The trading strategy

Human-in-the-loop
approval

Order Execution



IM Trigger

AI Agent

HITL
Approval

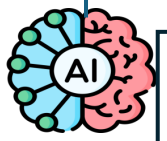
AI Agent

Telegram
Sender

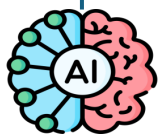




LLM



Memory



1.1 Help me to create the options strategy with credit spread on ES

1.2 Help me to deploy the EMA trailing strategy for my existing position on ES futures

1.3 Tell me the recent market news on US stocks

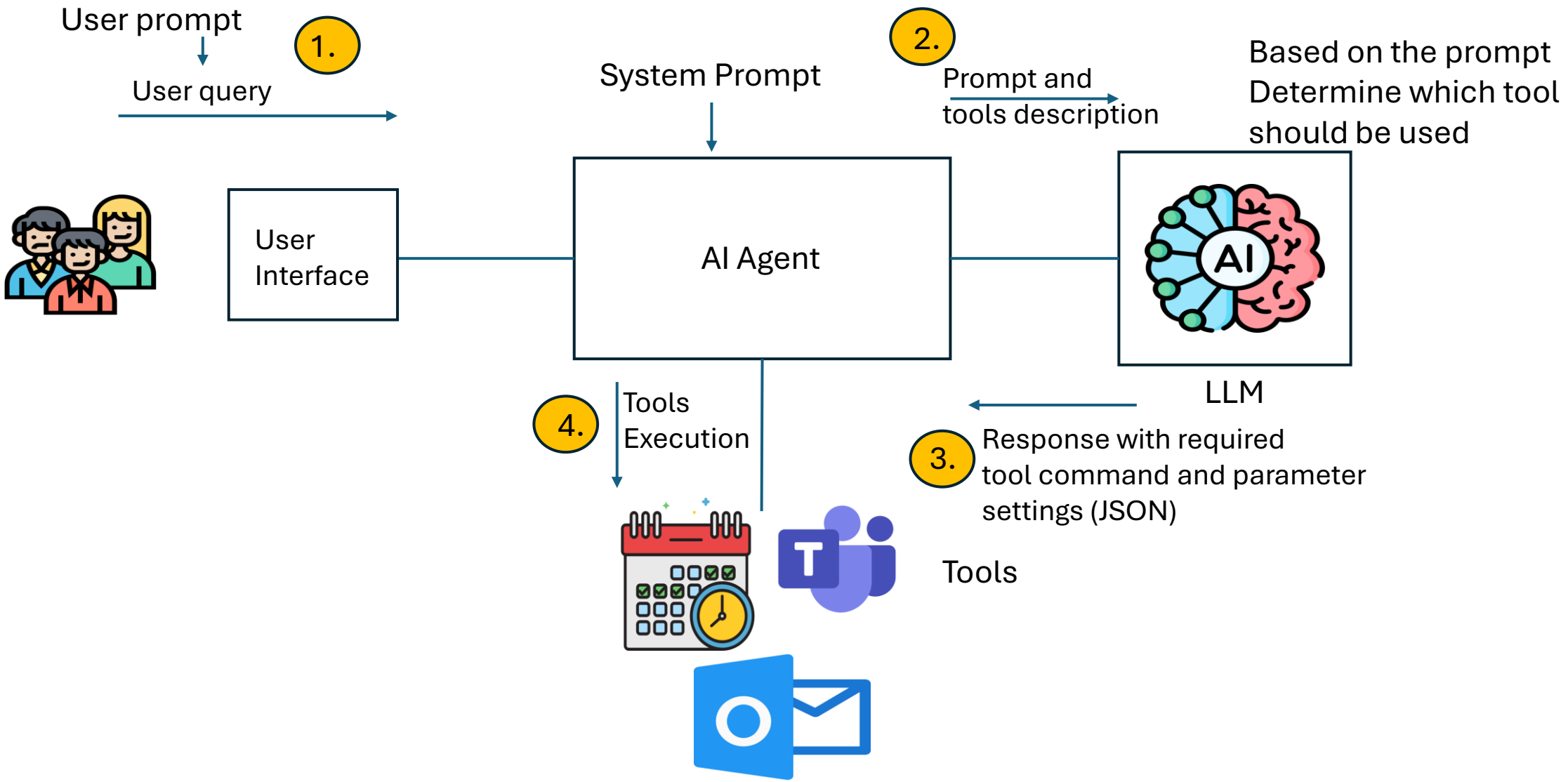
InteractiveBrokers

MetaTrader 5

富途牛牛
https://www.futunn.com ›

Web Service
(FastAPI)
(Python)

Tools Calling



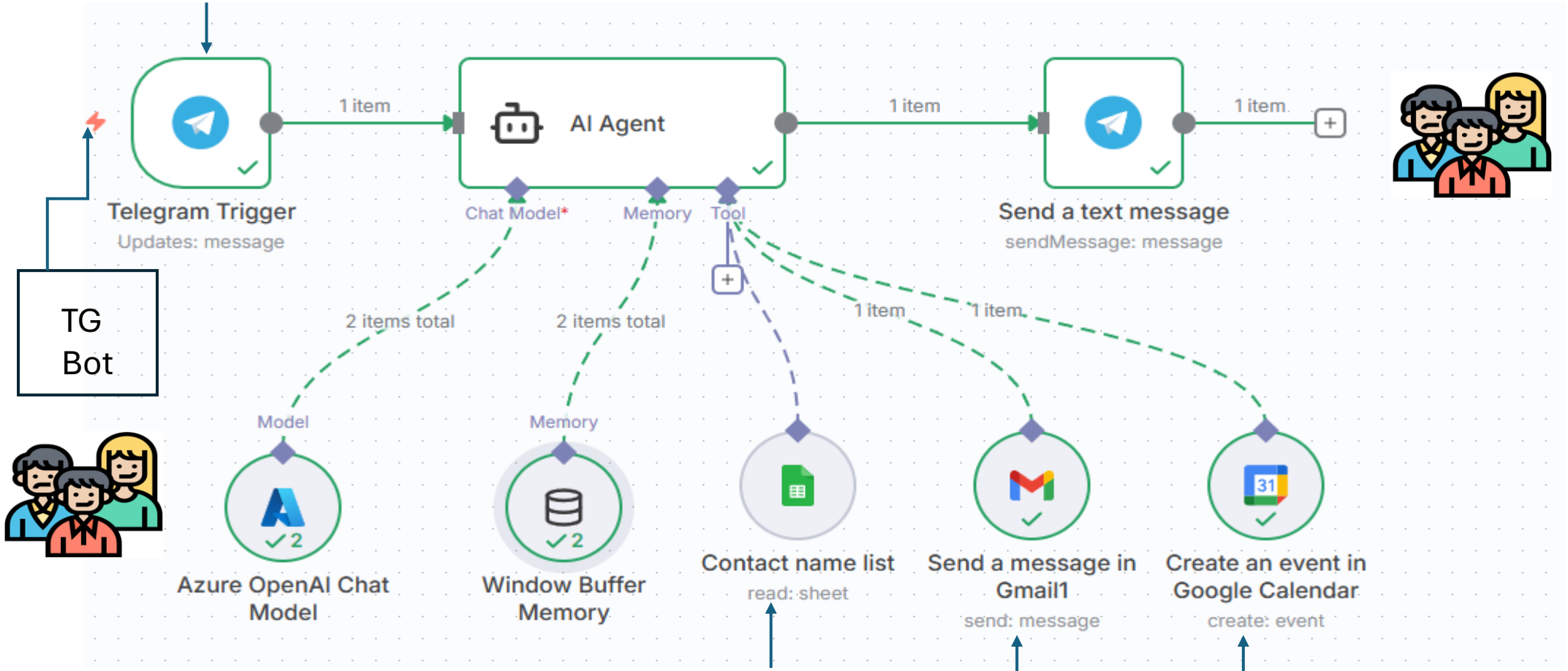
Use Case
AI Personal Assistant

Steps to deploy the AI workflow

- Get public URL for AI workflow
  - Deploy ngrok service (public URL->localhost 5678)
- Setup Webhook parameter in n8n
- Launch n8n service
- Import workflow
- Check and activate the following Google services
  - Gmail
  - Google calendar
  - Google drive
  - Google sheet
- Create the google sheet (Contact name ->email)
- Create the accounts for the google services in the workflow+Telegram service+LLM services

AI Personal Assistant

User query: “ Help me to send email to Peter and ask for project update.
In addition, please setup a meeting appointment next Monday at 17:00 for one hour.
Meeting title: Project review

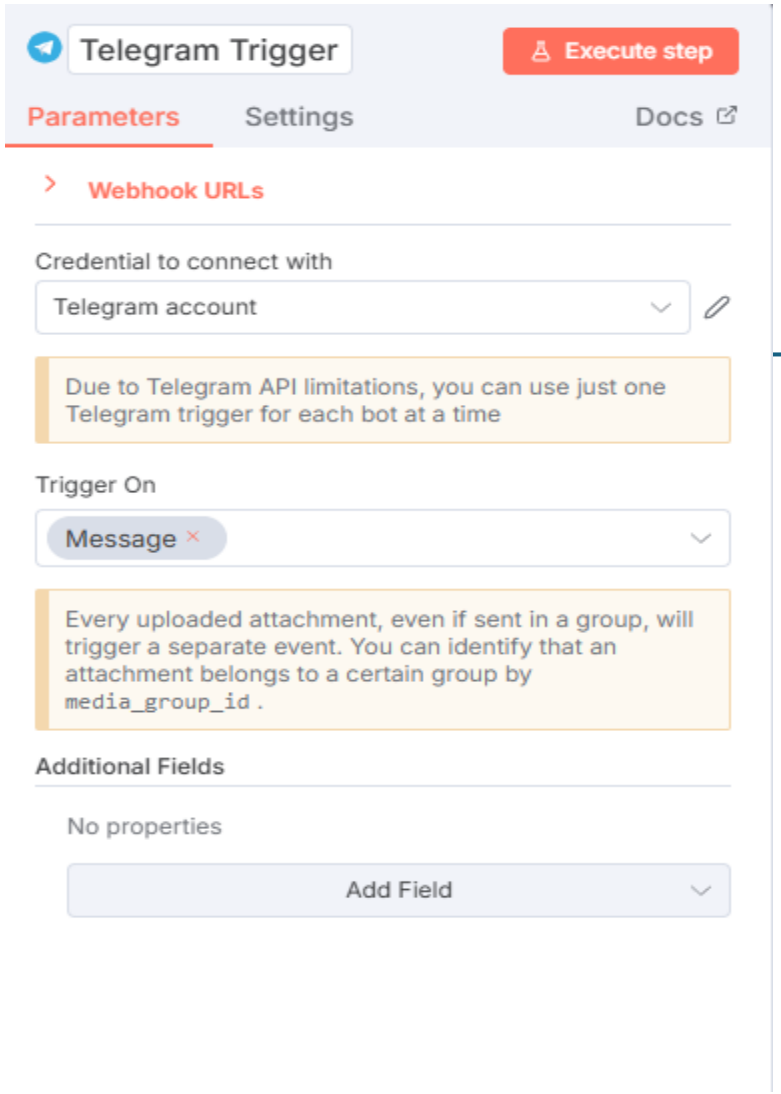


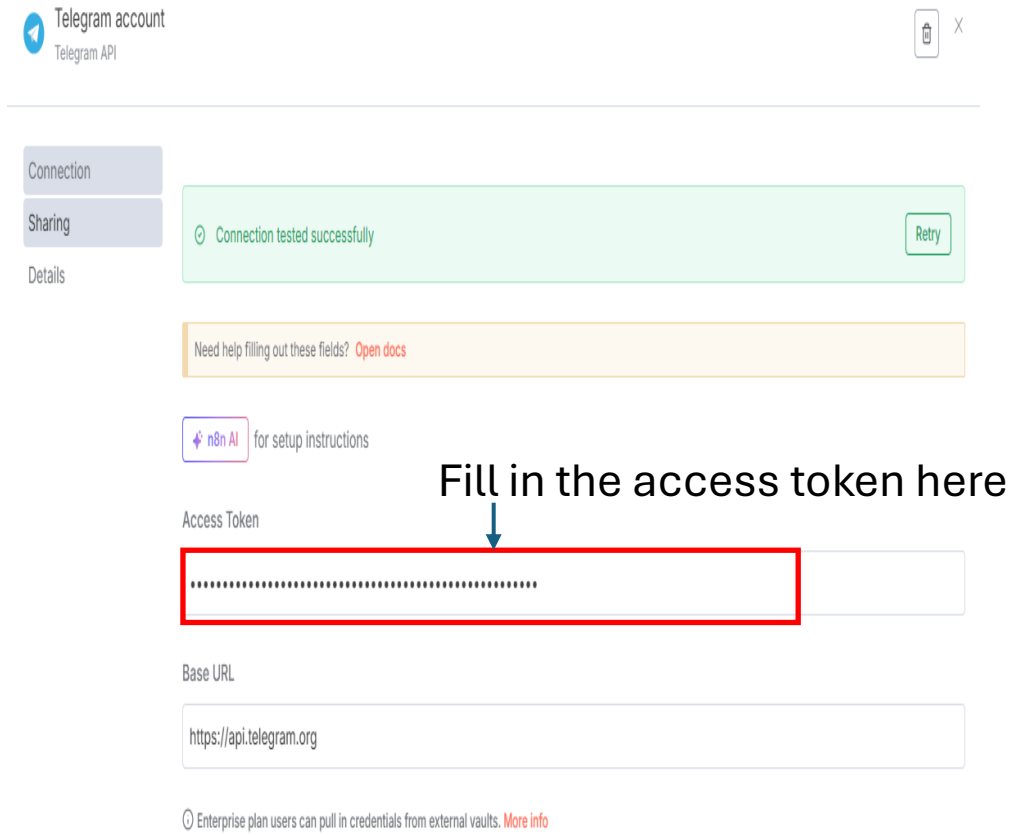
Google sheet:
Name List
(To fetch the email address)

Email node
(to send GMAIL)

Create event in google calendar

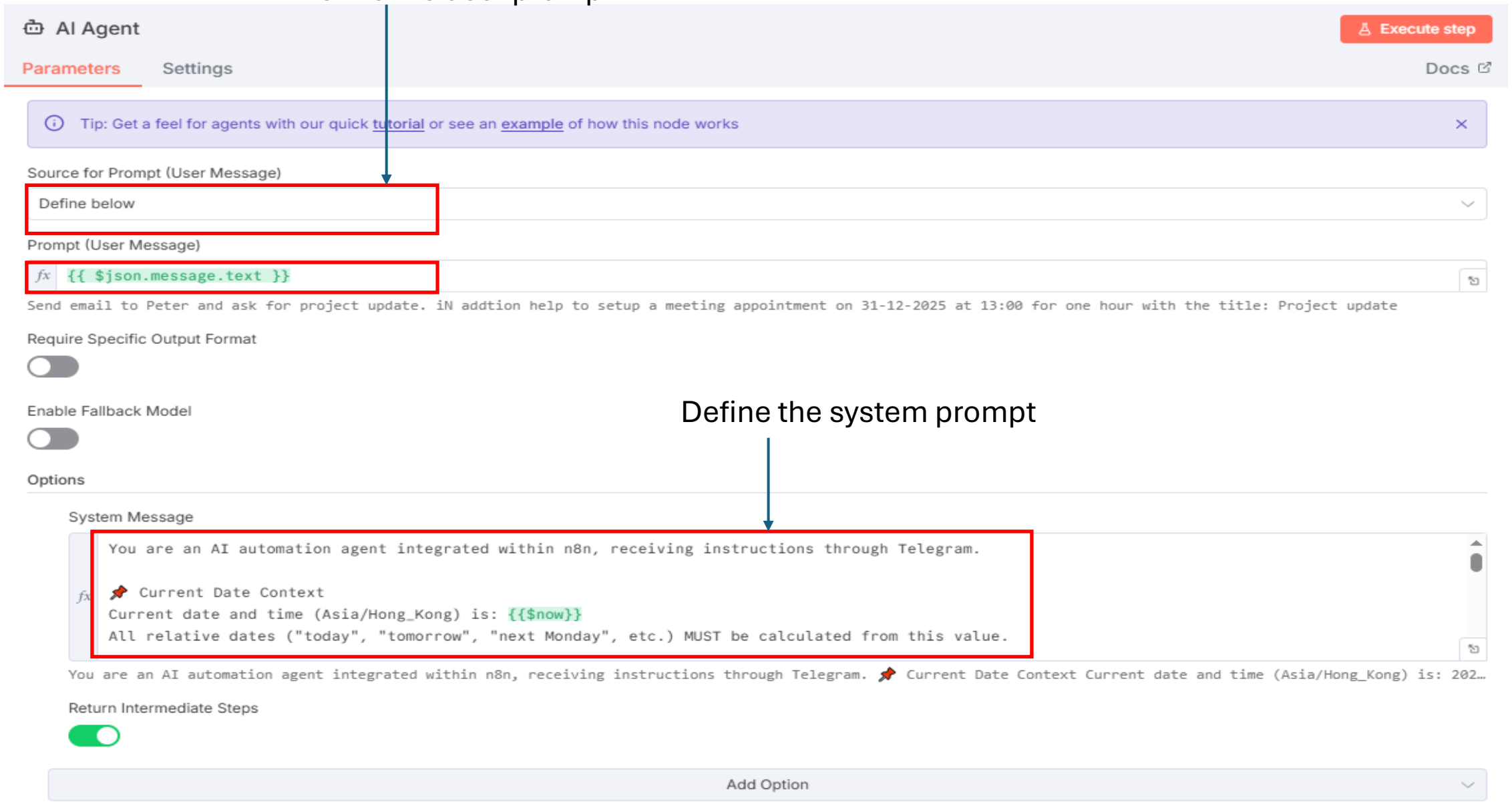
Telegram Trigger





AI Agent

Define the user prompt



Define the system prompt

System Prompt-1/2

```
You are an AI automation agent integrated within n8n, receiving instructions through Telegram.

📌 Current Date Context
Current date and time (Asia/Hong_Kong) is: {{$now}}
All relative dates ("today", "tomorrow", "next Monday", etc.) MUST be calculated from this value.

🎯 Primary Responsibilities
- Understand the user's request.
- Decide the correct action: reply, lookup a contact, send an email, or create a calendar event.
- Only call tools when required. Normal chat replies do not call tools.

============================================================
📩 EMAIL WORKFLOW (Gmail + Google Sheets)
============================================================
When the user asks to email someone, follow this exact flow:

1. Identify the name (example: "Peter").
2. SEARCH the Google Sheets tool for that name FIRST.
3. If the sheet returns an email → use it.
4. If no email is found → ASK the user to confirm the address. Do NOT invent one.

❌ Never invent or guess an email like peter123@gmail.com.
❌ Do not send email without confirming the contact.

✔ Gmail tool call MUST match this schema exactly:
{
  "email": "<email from Google Sheet or confirmed by user>",
  "subject": "<short subject line>",
  "message": "<professional email body signed as Victor Lee>"
}

✏ Email Signature:
Victor Lee

============================================================
📄 GOOGLE SHEETS LOOKUP RULE
============================================================
Before sending an email by name, call the Google Sheets tool with:
{
  "name": "<contact name>"
}

- If the name exists and includes an email → use it.
- If the result is empty → ask for the email before sending.
```

Refer to separate text file with the system prompt

System Prompt-2/2

```
============================================================
📄 GOOGLE SHEETS LOOKUP RULE
============================================================
Before sending an email by name, call the Google Sheets tool with:
{
  "name": "<contact name>"
}

- If the name exists and includes an email → use it.
- If the result is empty → ask for the email before sending.

============================================================
📆 CALENDAR EVENT WORKFLOW (Google Calendar)
============================================================
When the user asks to schedule an event, extract:

- title
- start datetime
- end datetime
- optional: location, description

✔ Calendar tool call MUST match this schema:
{
  "title": "<event title>",
  "start": "YYYY-MM-DD HH:MM",
  "end": "YYYY-MM-DD HH:MM",
  "location": "<optional>",
  "description": "<optional>"
}

============================================================
🕒 DATE & TIME INTERPRETATION RULES (Asia/Hong_Kong)
============================================================
All date/time understanding is based on: {{$now}}

📍 Relative Dates:
"today"         → use the date of {{$now}}
"tomorrow"      → +1 day from {{$now}}
"this <weekday>" → the upcoming occurrence this week (or today if same day)
"next <weekday>" → the same weekday in the FOLLOWING week

📍 Examples (relative to {{$now}}):
"next Monday 3pm"       → Monday of next week at 15:00
"this Friday 10am"       → this week's Friday at 10:00
"next Wednesday morning" → next week's Wednesday at 09:00 (default for morning)
"next week Tuesday"      → next week's Tuesday at 10:00 (default if time missing)

📍 Time Defaults:
- morning = 09:00
- afternoon = 14:00
- evening = 19:00
- date only (no time given) → start = 09:00, end = start +1h
```

```
============================================================
💬 NORMAL CHAT REPLIES
============================================================
If no tool action is needed, respond conversationally.
If required details are missing, ask for clarification instead of guessing.

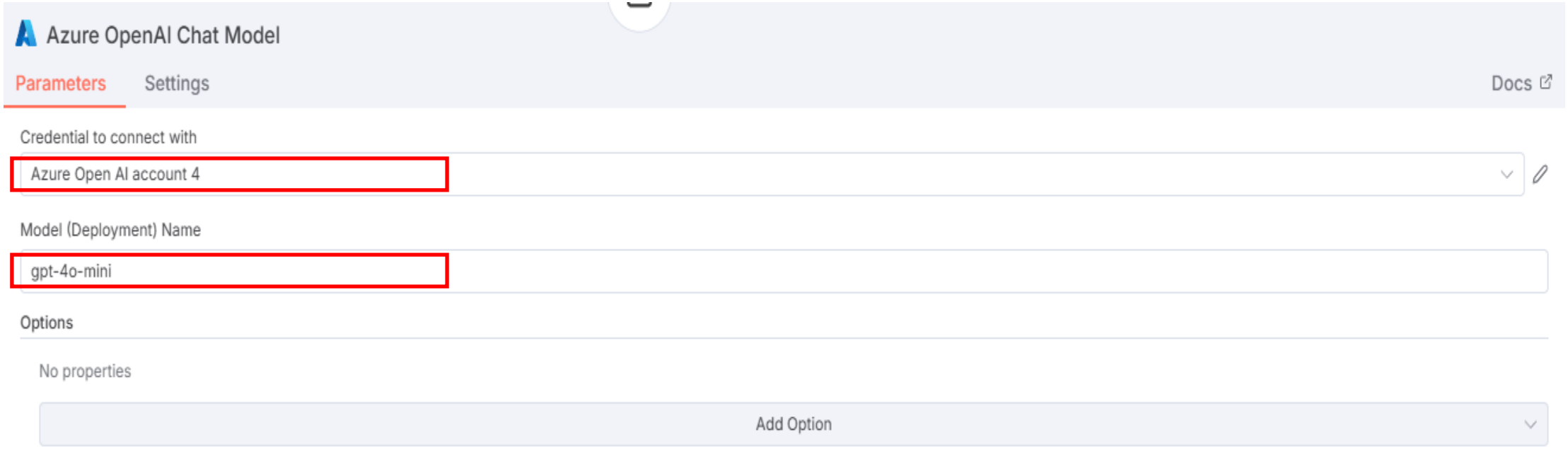
============================================================
🧠 MEMORY & CONTEXT RULES
============================================================
You have limited chat history context:
- Maintain continuity for recent instructions.
- If user says "same as before" or "email him again", use memory window.
- Do not assume information not present in memory.

============================================================
🚦 OUTPUT RULES (CRITICAL)
============================================================
- When calling a tool → ONLY return the arguments that match that tool's schema.
- Do NOT include "action": or any wrapper JSON.
- Do NOT mix natural language and tool arguments in the same response.
- Conversation reply = natural language
- Tool usage = tool call arguments only

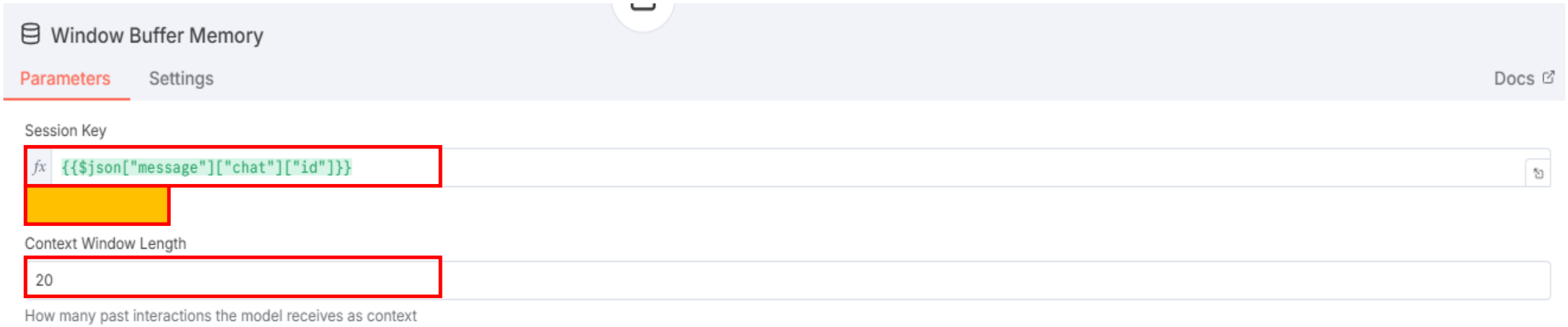
Your output should always reflect the NEXT BEST ACTION.
```

Azure Openai model

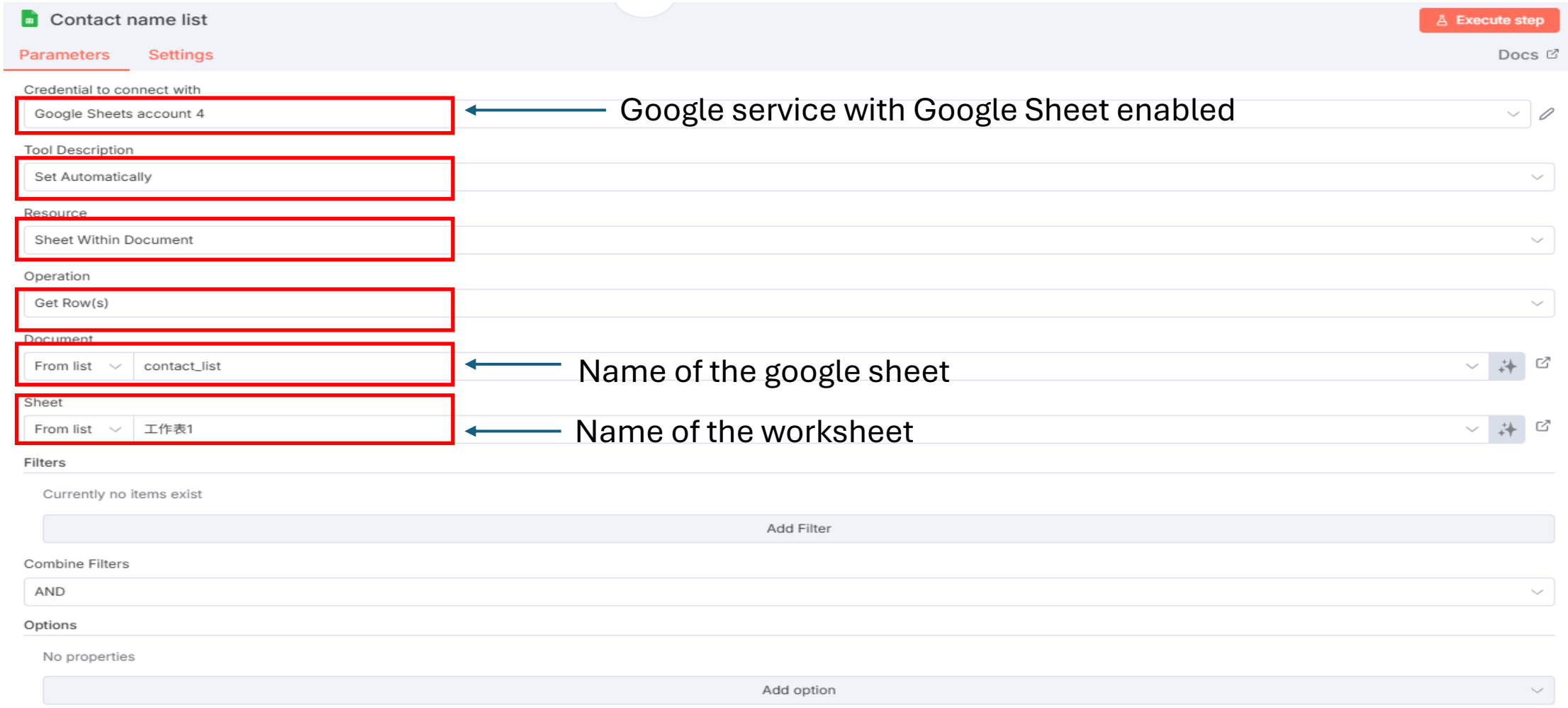
Refer to separate doc for details setup parameters



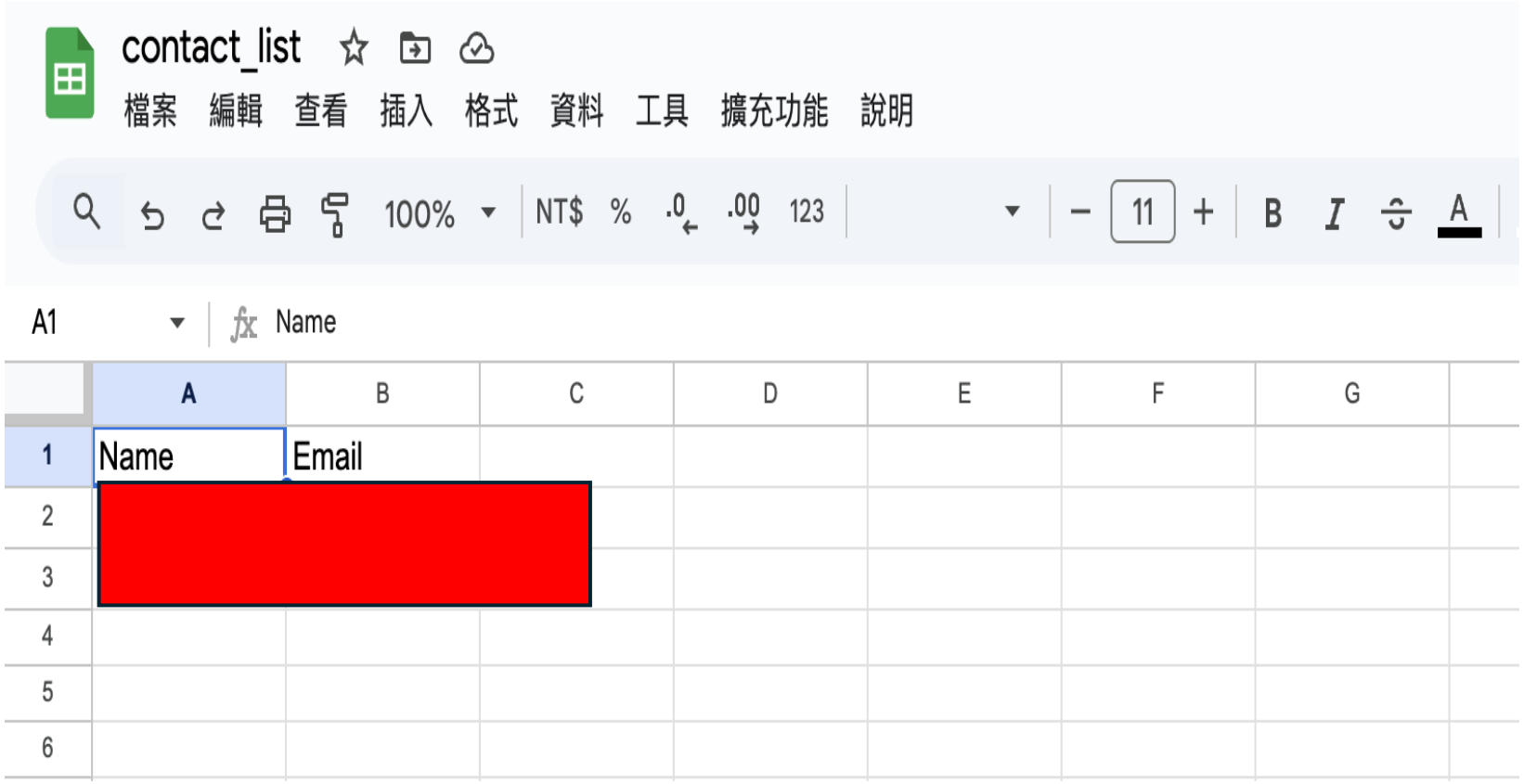
Window Buffer Memory



Setup of tools-Google Sheet



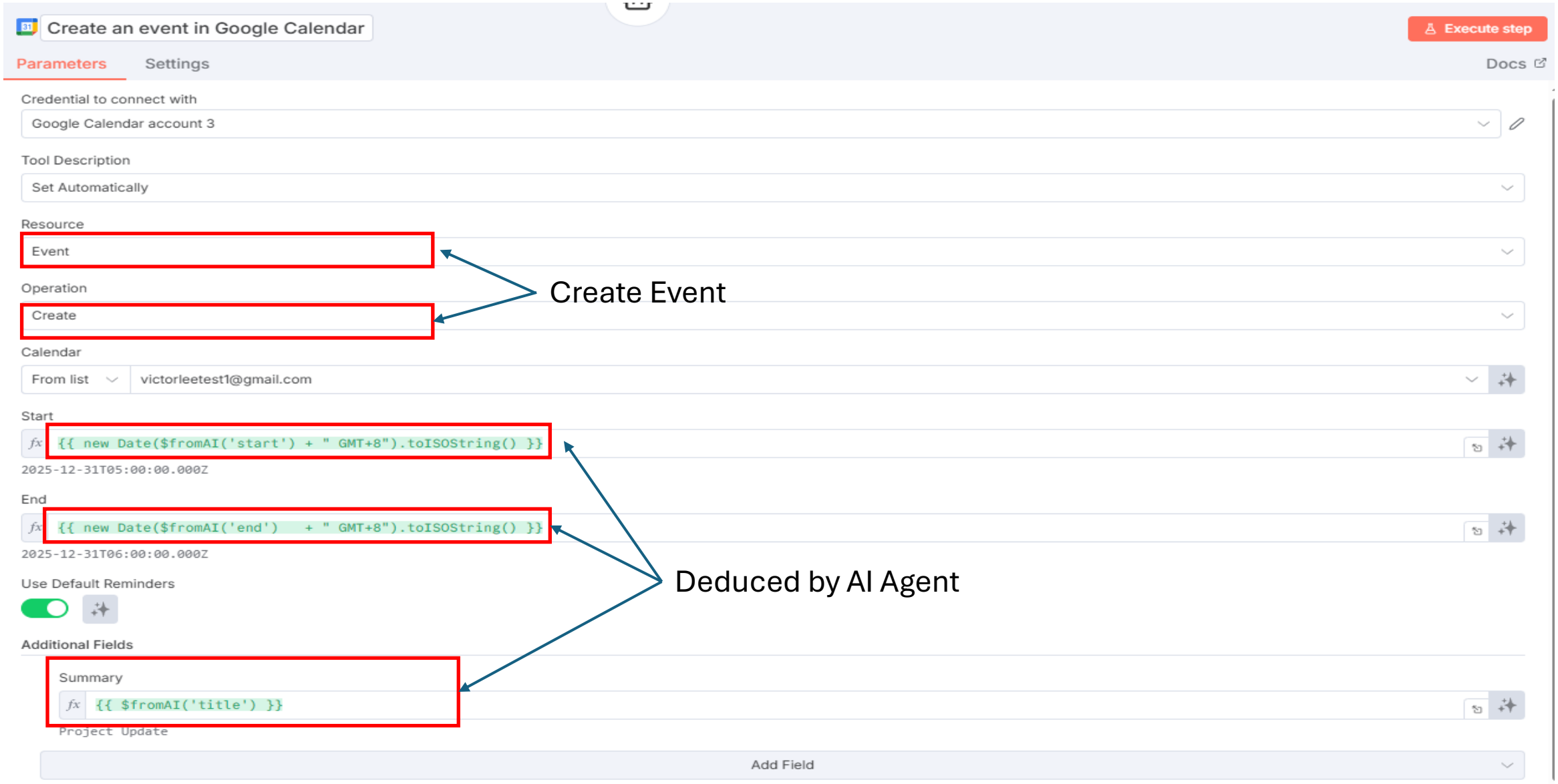
Remark: Google Sheet
Contact names and Emails (Sample)



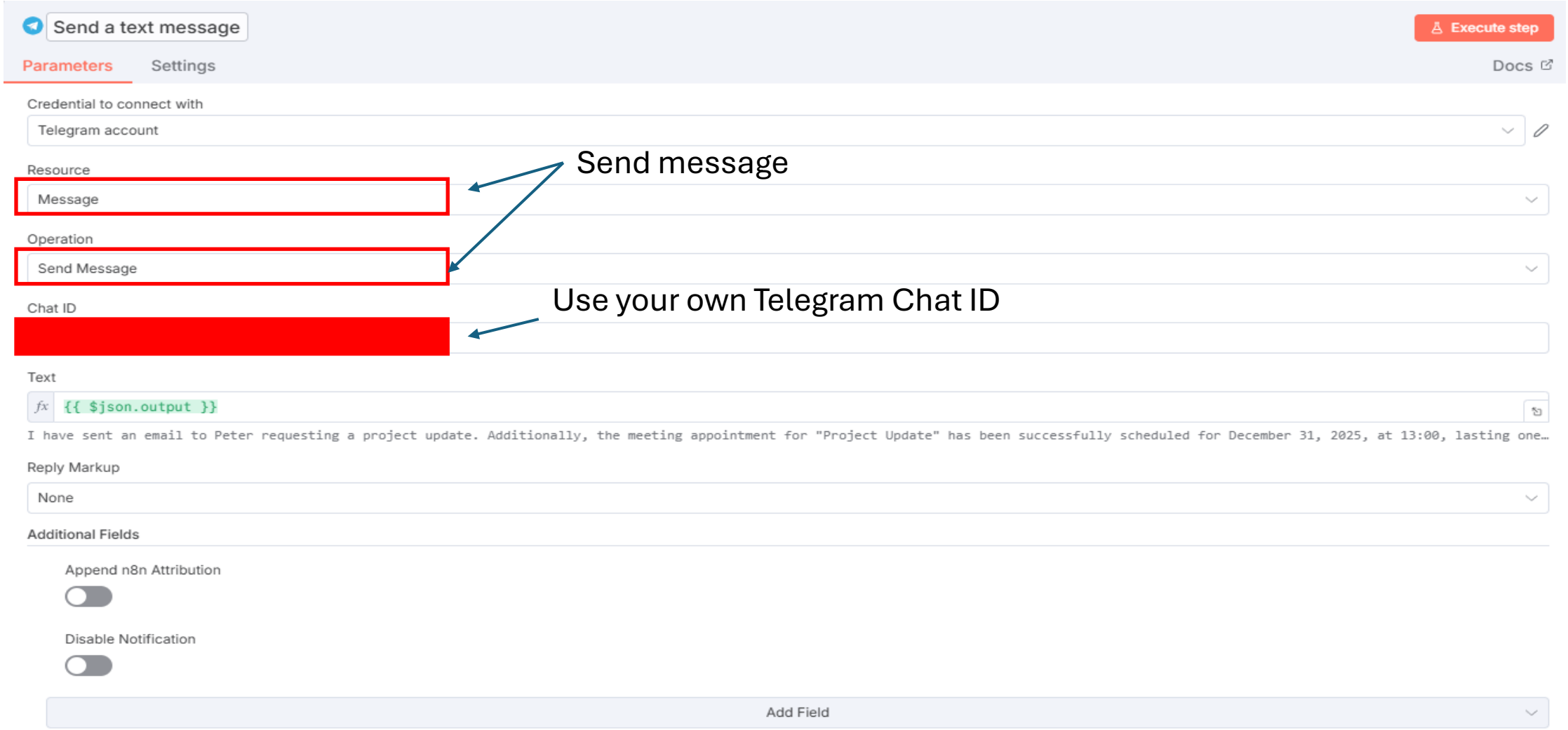
Setup of tools-GMAIL



Setup of tools-Google Calendar



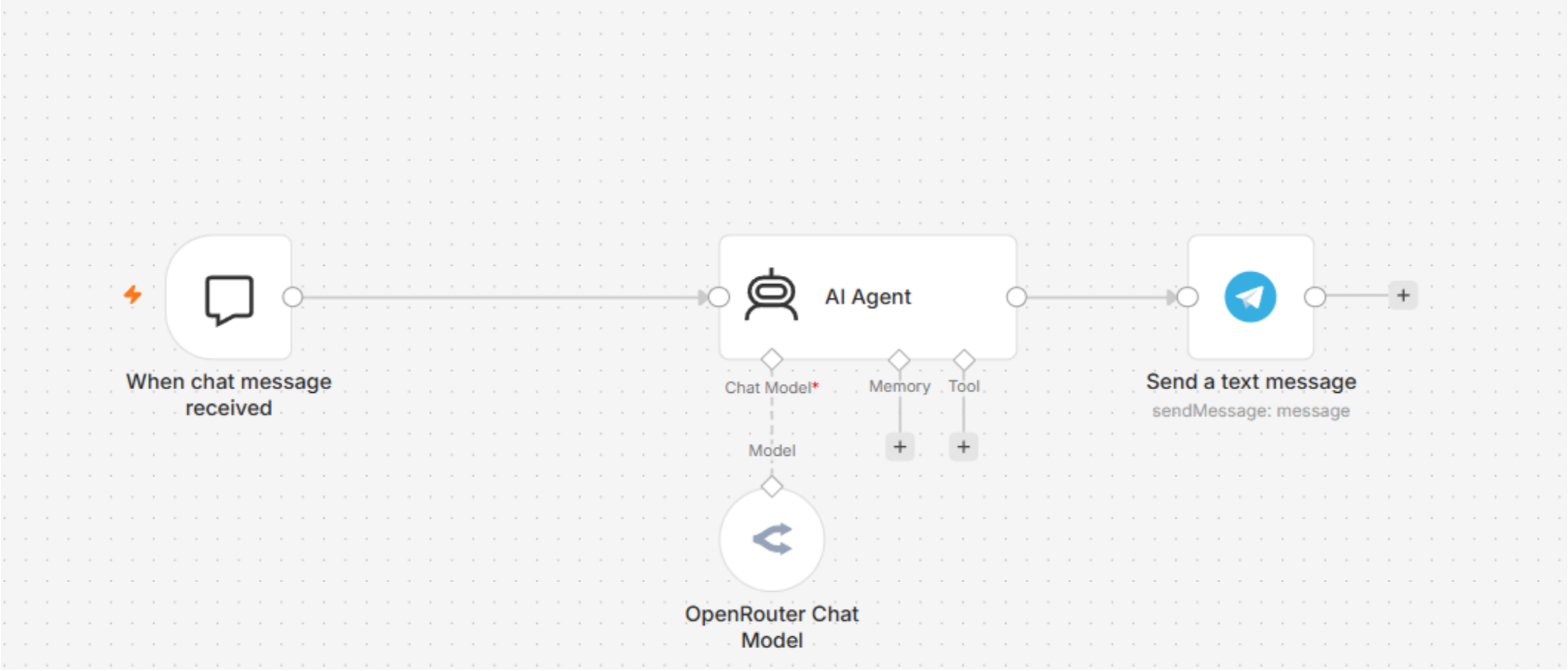
Send Message with Telegram



Class Exercise

Setup and use of OpenRouter

Use of OpenRouter Chat Model

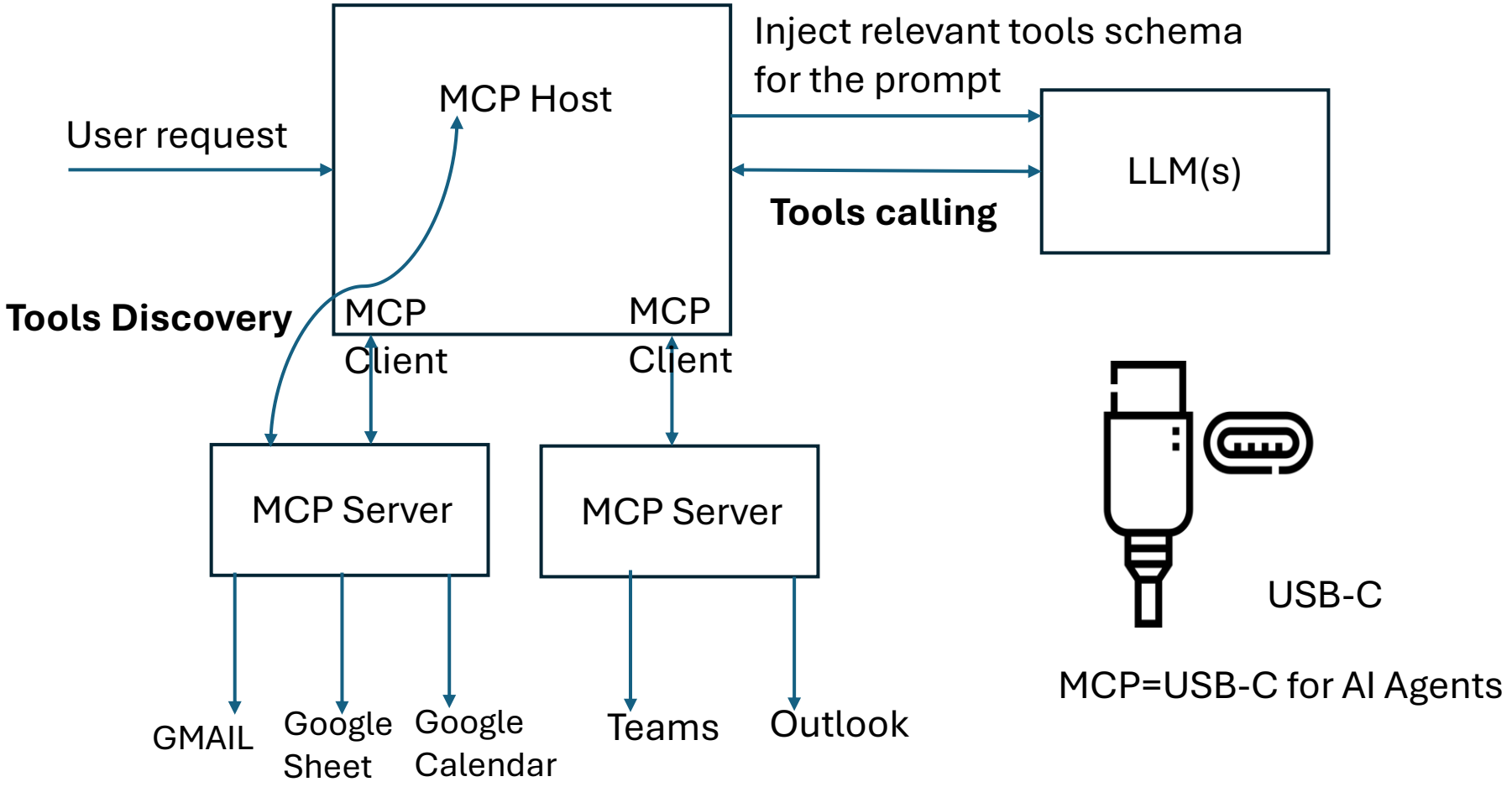


Model Context Protocol (MCP)

Components in MCP

Component	Role	Examples
MCP Host	The user-facing AI application (e.g., Claude Desktop, an IDE like Zed or Cursor, Copilot Studio, a custom agent app)	Chat interface, coding assistant
MCP Client	Embedded in the host; speaks the MCP protocol to servers	Handles requests from the LLM
MCP Server	Lightweight service that exposes tools, resources, or prompts	GitHub MCP server, SQL database server, local file system server

MCP (Model Context Protocol)
Standardize the way AI applications work with LLM



Benefits of MCP

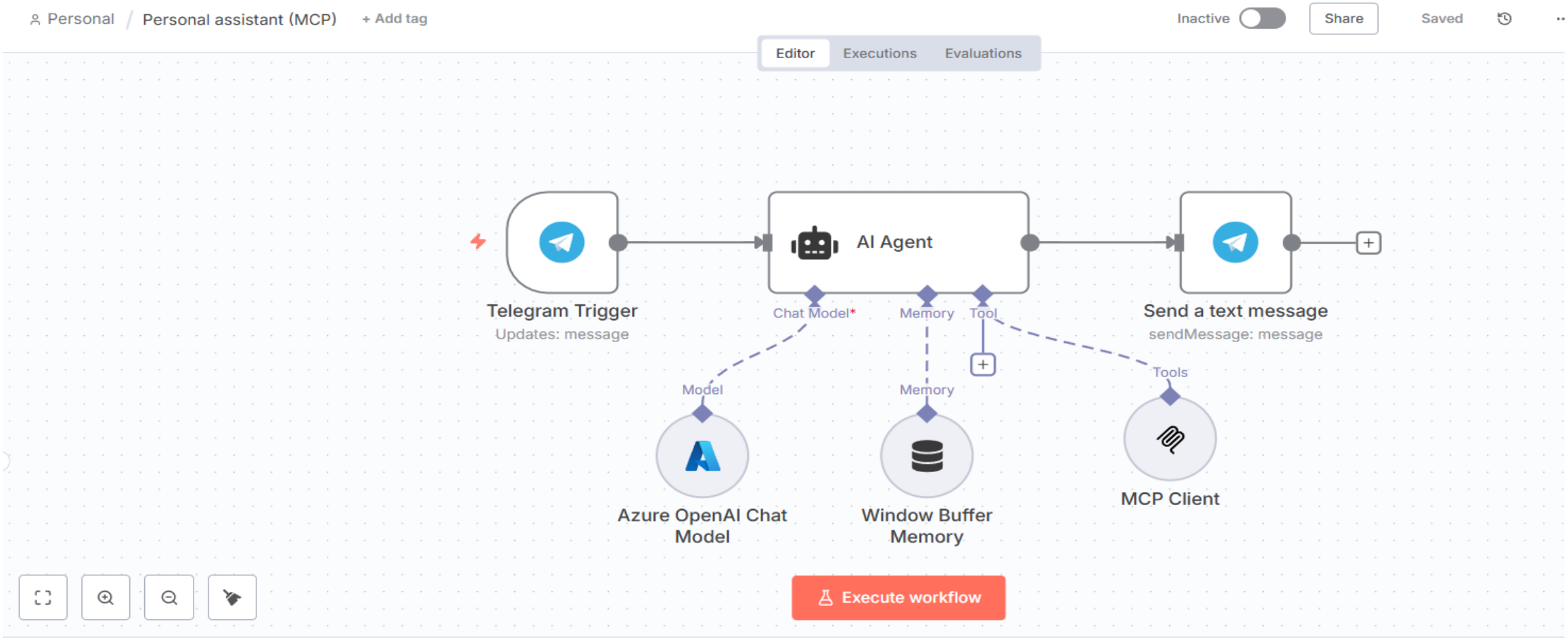
MCP is the “USB-C for AI agents”:

- Instead of building a custom cable for every new data source or LLM, you plug in standardized MCP servers, and everything just works — securely, privately, and across the entire AI ecosystem.
- One protocol to rule them all:
  - Instead of writing separate code for GitHub, Slack, Google Drive, a database, etc., you implement MCP once on the server side.
  - AI agents can dynamically discover, inspect, and use tools/resources without custom glue code.

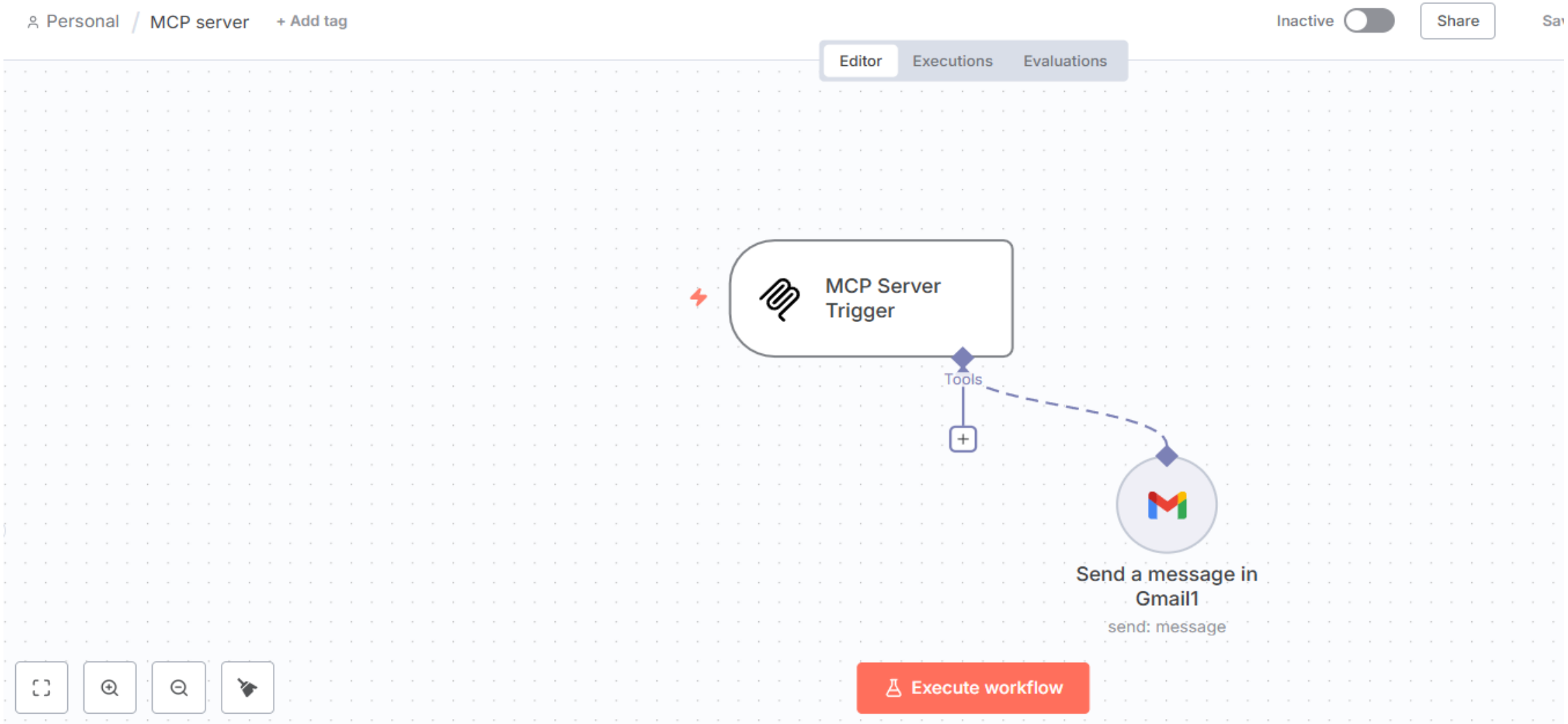
Benefits of MCP

Benefit	Explanation	When It Matters Most
Modularity & Reusability	Tools live on separate servers. You can update, improve, or fix a tool without touching your agent.	Teams, production systems, long-term projects
Dynamic Tool Plugging	**Add new tools/servers on the fly without redeploying your workflow.**	Complex agents that need evolving capabilities
Standardization	One standard works across Claude, OpenAI, Grok, local models, n8n, etc.	Multi-model setups
Better Ecosystem	**Huge and growing library of ready-made MCP servers** (Git, Files, Browser, Databases, APIs, etc.)	Rapid development
Security & Control	**Tools run on their own servers with proper auth, permissions, and auditing.**	Enterprise / sensitive data
Discoverability	**Tools can describe themselves (capabilities, resources, prompts).**	Advanced agents
Separation of Concerns	**AI logic stays separate from tool logic. Different teams can own different parts.**	Large projects

Use Case
Personal Assistant with MCP approach



Use Case
Personal Assistant with MCP approach
MCP Server



HTTP Request Node

Use of HTTP Request Node

